

Relatório 10: TLS em HTTP com Wireshark (VMs no VirtualBox)

Este relatório tem como objetivo montar um laboratório em NAT Network (rede NAT do VirtualBox) para demonstrar o uso de TLS no protocolo HTTP, comparando o tráfego em texto claro (HTTP sem TLS) e criptografado (HTTPS com TLS) e analisando as diferenças no Wireshark.

- Material de apoio: <https://github.com/vinlss/T02---Redes-de-Dados-1/>

I. Introdução

Nesta atividade, você configurará duas VMs (user 1 - Debian e user 2 - Debian) em uma NAT Network do VirtualBox para observar, via Wireshark no user 2 - Debian, como o TLS protege o conteúdo das mensagens HTTP. Em texto claro, o payload é legível; com TLS, apenas metadados e o handshake ficam visíveis.

O TLS cria um canal cifrado sobre TCP por meio do handshake, com negociação de chaves de sessão temporárias para proteger o tráfego de aplicação. Na prática, isso reduz a exposição a interceptação e leitura de conteúdo por terceiros no caminho.

Pilares abordados: foco em Confidencialidade (com menção a Autenticidade/Integridade via certificados e MAC do TLS).

II. Conceitos e Fundamentos

- HTTP x HTTPS: HTTPS = HTTP sobre TLS (camada de segurança entre transporte e aplicação).
- TLS (Handshake): ClientHello → ServerHello (+ certificado) → key exchange → chaves de sessão → comunicação criptografada.
- Certificados: neste lab usaremos certificado autoassinado no Apache.
- Wireshark: filtros úteis http, tls, tcp.port == 443, tls.handshake.

III. Ambiente e Topologia

VirtualBox — NAT Network: NatNetwork

Atenção: neste relatório usamos NAT Network (rede compartilhada entre VMs). Isso é diferente de NAT simples por VM (Attached to: NAT), que normalmente não permite comunicação direta entre as VMs.

1) Resumo do laboratório

Elemento	Definição
Rede do VirtualBox	NAT Network NatNetwork
user 1 - Debian	Servidor Apache, usando as portas 80 e 443
user 2 - Debian	Cliente de testes com curl e Wireshark
Adaptadores das VMs	Uma placa por VM, ligada à NAT Network
Recursos das VMs	6 GB de RAM e 3 núcleos de CPU

2) Dados de referência usados no relatório

Item	Uso
Senha das máquinas virtuais	pytest
Senha solicitada pelo sudo	pytest
<IP_USER1>	IP atual do user 1 - Debian
<IP_USER2>	IP atual do user 2 - Debian

3) Pacotes usados no laboratório

- user 1 - Debian: apache2, openssl
- user 2 - Debian: curl, wireshark

4) Guia auxiliar de VirtualBox

Se não estiver conseguindo copiar e colar comandos entre o computador host e a VM, redimensionar a tela corretamente ou usar melhor a integração com o VirtualBox, consulte o guia [VirtualBox Guest Additions.md](#) para instalar o VirtualBox Guest Additions.

IV. Instalação e Preparação

Substitua as ocorrências de <IP_USER1> e <IP_USER2> pelos endereços obtidos nas suas VMs.

1) Preparar a rede no VirtualBox

1. Abrir o gerenciador de redes NAT:
 - No VirtualBox, clique em “Arquivos”
 - Clique em “Ferramentas”
 - Clique em “Gerenciador de Rede”
 - Clique em “Redes NAT”

2. Se já existir NatNetwork, apenas confira se está configurada:

- Name: NatNetwork
- IPv4 Prefix: verifique o prefixo (ex.: 10.0.2.0/24)
- Habilitar DHCP: Marcado

3. Se NÃO existir NatNetwork: criar uma:

- Clique em “Criar”
- Name: NatNetwork
- IPv4 Prefix: ex.: 10.0.2.0/24 (padrão)
- Marque Habilitar DHCP
- Confirme com Aplicar

The screenshot shows the Oracle VM VirtualBox Network Manager interface. At the top, there are three tabs: 'Redes Exclusivas de Hospedeiro (Host-only)', 'Redes NAT', and 'Redes de Nuvem'. The 'Redes NAT' tab is selected. Below the tabs, there is a table with the following columns: 'Nome', 'Prefixo IPv4', 'Prefixo IPv6', and 'Servidor DHCP'. The table contains one entry: 'NatNetwork' with '10.0.2.0/24' for IPv4, 'fd17:625c:f037:2::/64' for IPv6, and 'Habilitado' for the DHCP server. Below the table, there are two tabs: 'Opções Gerais' and 'Redirecionamento de Portas'. The 'Opções Gerais' tab is selected. It shows the following configuration: 'Nome: NatNetwork', 'Prefixo IPv4: 10.0.2.0/24', a checked checkbox for 'Habilitar DHCP', an unchecked checkbox for 'Habilitar IPv6 (E)', 'Prefixo IPv6: fd17:625c:f037:2::/64', and an unchecked checkbox for 'Divulgar Rota IPv6 Padrão'. At the bottom right, there are two buttons: 'Aplicar' and 'Desfazer'.

Nome	Prefixo IPv4	Prefixo IPv6	Servidor DHCP
NatNetwork	10.0.2.0/24	fd17:625c:f037:2::/64	Habilitado

Opções Gerais Redirecionamento de Portas

Nome: NatNetwork

Prefixo IPv4: 10.0.2.0/24

☒ Habilitar DHCP

☐ Habilitar IPv6 (E)

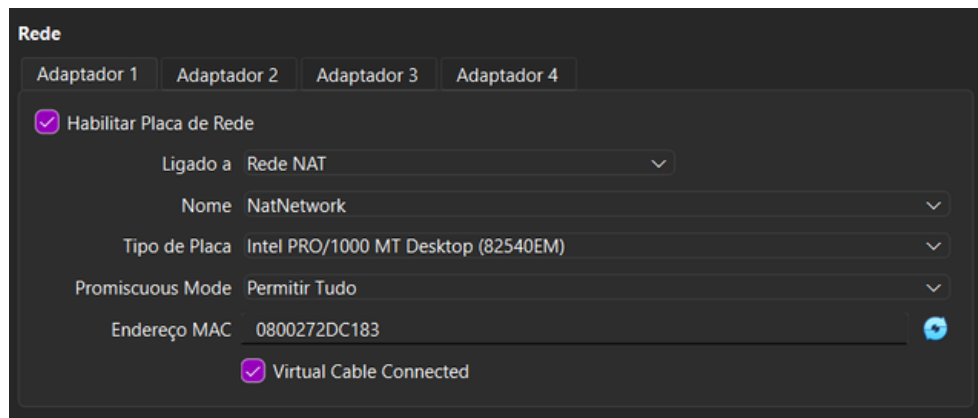
Prefixo IPv6: fd17:625c:f037:2::/64

☐ Divulgar Rota IPv6 Padrão

Aplicar Desfazer

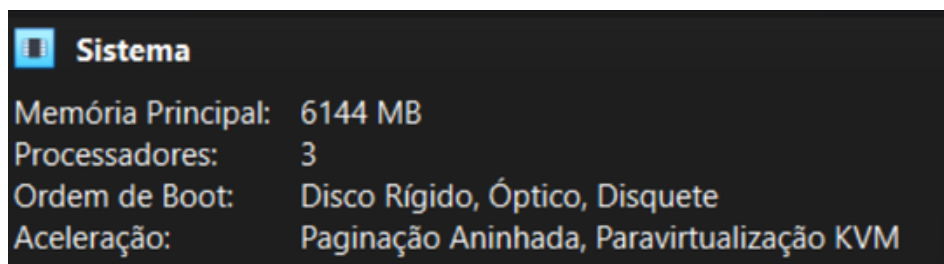
4. Vincular cada VM à NatNetwork:

- Para user 1 - Debian e user 2 - Debian → Configurações (Settings) → Rede (Network)
- Adaptador 1 (Adapter 1) → Conectado a: NAT Network → Nome: NatNetwork
- OK



5. Ajustar recursos de hardware das VMs (user 1 - Debian e user 2 - Debian):

- VirtualBox → Configurações → Sistema → Placa-mãe → Memória Base: 6144 MB (6 GB)
- VirtualBox → Configurações → Sistema → Processador → Processadores: 3
- Após ajustar os recursos, iniciar as VMs



Ao final desta etapa: as duas VMs devem estar ligadas, vinculadas à NatNetwork e prontas para receber IP via DHCP.

2) Conferir IPs e validar a comunicação

1. Em cada VM Debian, execute:

comando: ip a

2. Identifique o endereço IPv4 usado na interface conectada à NatNetwork e verifique se os IPs de user 1 - Debian e user 2 - Debian pertencem à mesma faixa de rede configurada no VirtualBox.

- Exemplo: se a NatNetwork estiver em 10.0.2.0/24, os dois IPs devem seguir o padrão 10.0.2.x/24.
- Anote o IP do servidor como <IP_USER1> e o IP do cliente como <IP_USER2>.

```
User 1 user@vbox: ~
user@vbox: $ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:2d:c1:83 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 598sec preferred_lft 598sec
user@vbox: $

User 2 user@vbox: ~
user@vbox: $ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:7d:3c:21 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.6/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 585sec preferred_lft 585sec
user@vbox: $
```

Anote antes de seguir: todos os comandos posteriores usam esses dois valores. Se necessário, mantenha os IPs anotados em separado durante a execução do laboratório.

3. Se os IPs estiverem compatíveis, avance para o teste de comunicação.
4. Se os IPs não estiverem na mesma rede, ou se alguma VM não tiver recebido um IPv4 válido da NatNetwork, execute em cada VM Debian:

comando: `IFACE=$(ip route | awk '/default/ {print $5; exit}')`
`sudo dhclient -r "$IFACE" || true`
`sudo ip addr flush dev "$IFACE"`
`sudo dhclient "$IFACE"`
`ip -4 a show "$IFACE"`

Nota: a interface padrão em Debian no VirtualBox é normalmente enp0s3 (Ethernet, barramento 0, slot 3). Se precisar verificar ou consultar manualmente, execute `ip route` ou `ip a` para confirmar qual interface está ativa.

Após isso, execute novamente `ip a` nas duas VMs e confirme se os endereços agora estão na mesma faixa de rede.

5. Testar comunicação entre as VMs com ping:

No user 2 - Debian, execute:

comando: `ping -c2 <IP_USER1>`

No user 1 - Debian, execute:

`ping -c2 <IP_USER2>`

```
User 1 user@vbox: ~
user@vbox: $ ping -c 5 10.0.2.6
PING 10.0.2.6 (10.0.2.6) 56(84) bytes of data:
64 bytes from 10.0.2.6: icmp_seq=1 ttl=64 time=0.611 ms
64 bytes from 10.0.2.6: icmp_seq=2 ttl=64 time=0.860 ms
64 bytes from 10.0.2.6: icmp_seq=3 ttl=64 time=0.658 ms
64 bytes from 10.0.2.6: icmp_seq=4 ttl=64 time=0.598 ms
64 bytes from 10.0.2.6: icmp_seq=5 ttl=64 time=0.608 ms

--- 10.0.2.6 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4078ms
rtt min/avg/max/mdev = 0.598/0.667/0.860/0.098 ms
user@vbox: $

User 2 user@vbox: ~
user@vbox: $ ping -c 5 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.44 ms
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=1.12 ms
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=0.908 ms
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=1.04 ms
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=1.41 ms

--- 10.0.2.15 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4350ms
rtt min/avg/max/mdev = 0.908/1.182/1.436/0.206 ms
user@vbox: $
```

Ao final desta etapa: as duas VMs devem estar na mesma rede e responder ao ping entre si.

3) Preparar o servidor — user 1 - Debian

A) Instalar dependências

comando: `sudo apt update && sudo apt install -y apache2 openssl`

```
user@vbox:~$ sudo apt update && sudo apt install -y apache2 openssl
Hit:1 http://security.debian.org/debian-security bookworm-security InRelease
Hit:2 http://deb.debian.org/debian bookworm InRelease
Hit:3 http://deb.debian.org/debian bookworm-updates InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
271 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apache2 is already the newest version (2.4.67-1~deb12u2).
openssl is already the newest version (3.0.19-1~deb12u2).
0 upgraded, 0 newly installed, 0 to remove and 271 not upgraded.
user@vbox:~$
```

B) Criar página de teste com conteúdo identificável

comando: `sudo mkdir -p /var/www/html`
`echo "<h1>HELLO_TLS_HTTP</h1>" | sudo tee /var/www/html/index.html`

```
user@vbox:~$ sudo mkdir -p /var/www/html
echo "<h1>HELLO_TLS_HTTP</h1>" | sudo tee /var/www/html/index.html
<h1>HELLO_TLS_HTTP</h1>
user@vbox:~$
```

C) Ativar o Apache e verificar a porta HTTP

comando: `sudo systemctl enable --now apache2`
`ss -tulpn | grep :80`

Resultado esperado: deve aparecer uma linha indicando que o Apache está em estado LISTEN na porta 80.

```
user@vbox:~$ sudo systemctl enable --now apache2
ss -tulpn | grep :80
Synchronizing state of apache2.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable apache2
tcp    LISTEN 0      511      *:80      *.*
user@vbox:~$
```

Ao final desta etapa: o servidor deve estar respondendo em HTTP na porta 80.

4) Preparar o cliente — user 2 - Debian

A) Instalar curl e Wireshark

Observação: escolha "Yes" na configuração do Wireshark

comando: `sudo apt update`

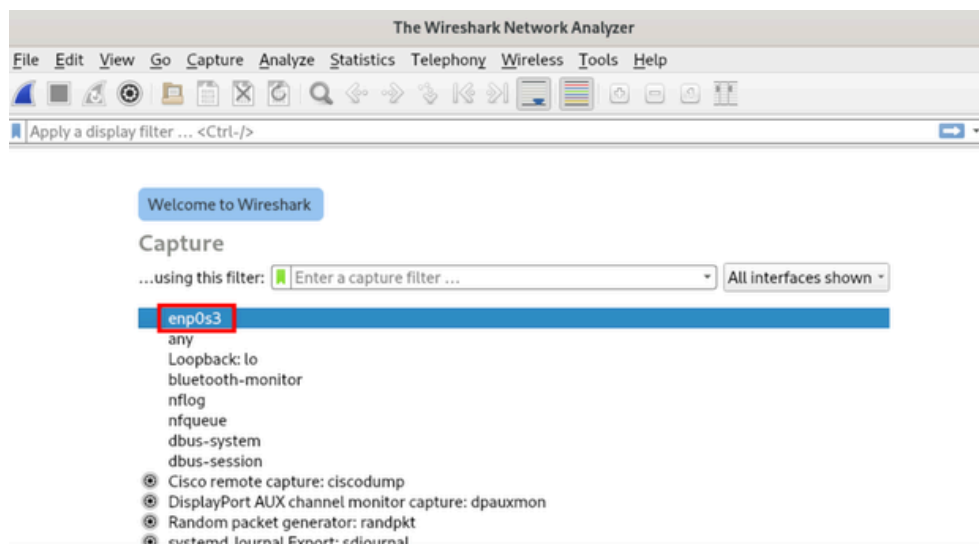
`sudo apt install -y curl wireshark`

```
user@vbox:~$ sudo apt install -y curl wireshark
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (7.88.1-10+deb12u14).
wireshark is already the newest version (4.0.17-0+deb12u3).
0 upgraded, 0 newly installed, 0 to remove and 281 not upgraded.
user@vbox:~$
```

B) Abrir o Wireshark e identificar a interface de captura

comando: `sudo wireshark`

Esse comando apenas abre o programa. Na tela inicial, identifique a interface da NAT Network (a que mostra o IP do user 2 - Debian, costuma ser `enp0s3`) e deixe o Wireshark pronto para iniciar a captura no próximo passo.



Durante a análise dos pacotes, use como referência o filtro:

filtro: `ip.addr == <IP_USER1>`

Ao final desta etapa: o cliente deve estar com curl instalado, o Wireshark aberto e a interface correta já identificada.

Observação: os filtros indicados neste relatório são filtros de exibição no Wireshark, usados para localizar melhor os pacotes já capturados.

V. Procedimentos (Passo a Passo)

Checklist antes dos testes

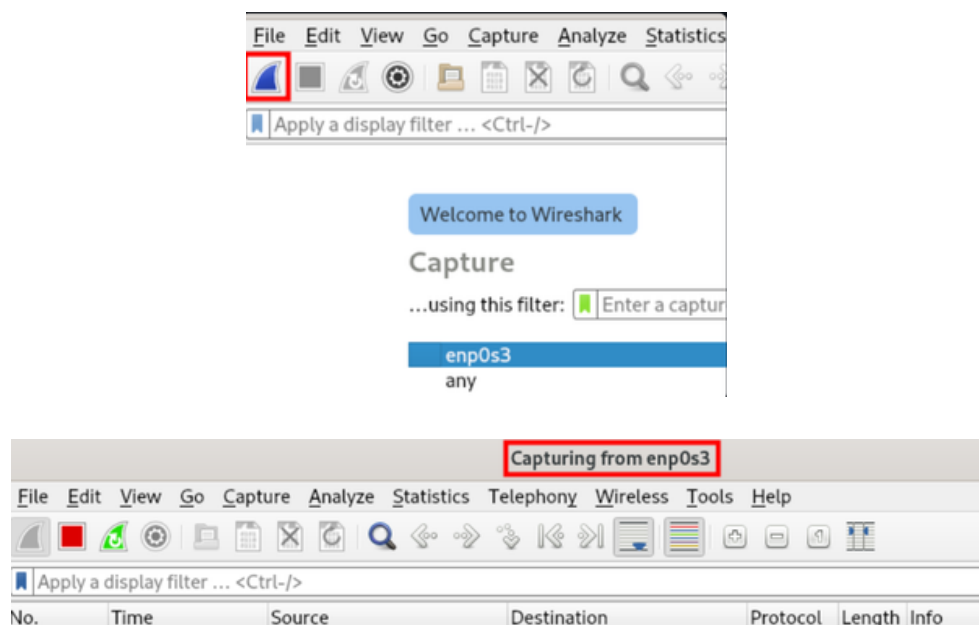
Antes de iniciar os cenários, confirme:

- o IP do servidor está anotado como <IP_USER1>;
- o Apache responde em HTTP na porta 80;
- o Wireshark está aberto no user 2 - Debian;
- a interface da NAT Network já foi identificada (costuma ser enp0s3).

Cenário 1 — HTTP sem TLS (porta 80)

A) Preparar a captura

1. No user 2 - Debian: com o Wireshark já aberto, inicie a captura na interface da NAT Network identificada anteriormente.



1. Observação: o quadrado vermelho indica que a captura está ativa.

B) Executar o teste HTTP

1. No user 2 - Debian: execute:

comando: `curl -v http://<IP_USER1>`

Resultado esperado no terminal: o curl deve completar a requisição e exibir a resposta do Apache, incluindo o conteúdo da página com HELLO_TLS_HTTP.

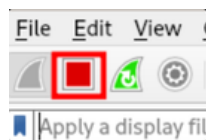

```

user@vbox:~$ curl -v http://10.0.2.15
* Trying 10.0.2.15:80...
* Connected to 10.0.2.15 (10.0.2.15) port 80 (#0)
> GET / HTTP/1.1
> Host: 10.0.2.15
> User-Agent: curl/7.88.1
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Thu, 07 May 2026 23:21:23 GMT
< Server: Apache/2.4.67 (Debian)
< Last-Modified: Thu, 07 May 2026 23:06:49 GMT
< ETag: "18-6514255d209f2"
< Accept-Ranges: bytes
< Content-Length: 24
< Content-Type: text/html
<
<h1>HELLO_TLS_HTTP</h1>
* Connection #0 to host 10.0.2.15 left intact
user@vbox:~$

```

C) Observar no Wireshark

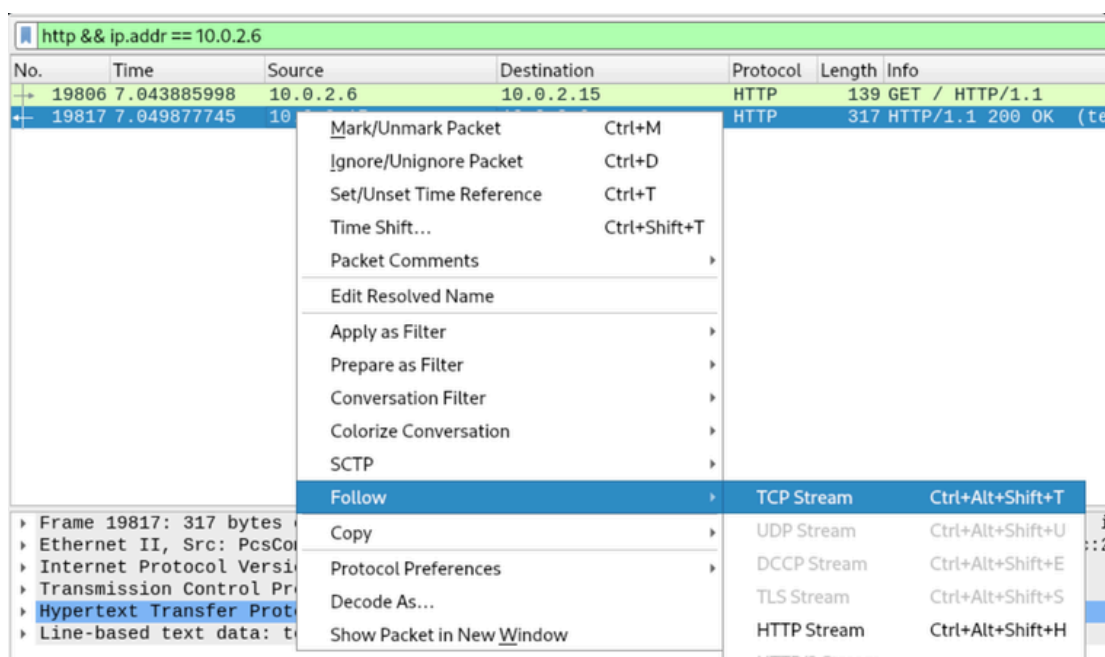
1. Pause a captura no Wireshark.

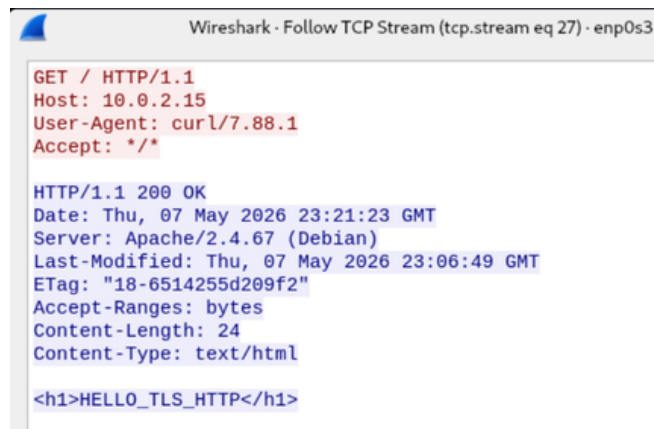


2. Aplique o filtro:

filtro: http && ip.addr == <IP_USER1>

3. O que observar: requisição GET / HTTP/1.1 e resposta 200 OK com payload legível (HTML contendo HELLO_TLS_HTTP).



A screenshot of the Wireshark interface showing a packet capture of an HTTP transaction. The top bar indicates 'Wireshark · Follow TCP Stream (tcp.stream eq 27) · enp0s3'. The packet list on the left shows a single packet. The packet details pane on the right shows the structure of the HTTP message. The request is a GET / HTTP/1.1 from host 10.0.2.15 using curl/7.88.1. The response is an HTTP/1.1 200 OK from an Apache/2.4.67 (Debian) server, dated Thu, 07 May 2026 23:21:23 GMT. The response body contains the HTML text '<h1>HELLO_TLS_HTTP</h1>'.

```
GET / HTTP/1.1
Host: 10.0.2.15
User-Agent: curl/7.88.1
Accept: */*

HTTP/1.1 200 OK
Date: Thu, 07 May 2026 23:21:23 GMT
Server: Apache/2.4.67 (Debian)
Last-Modified: Thu, 07 May 2026 23:06:49 GMT
ETag: "18-6514255d209f2"
Accept-Ranges: bytes
Content-Length: 24
Content-Type: text/html

<h1>HELLO_TLS_HTTP</h1>
```

Ao final deste cenário: o tráfego HTTP deve aparecer em texto claro no Wireshark.

Cenário 2 — HTTP com TLS (HTTPS na porta 443)

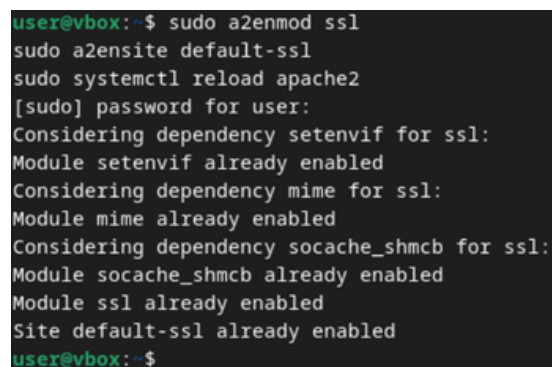
A) Preparar HTTPS no servidor — user 1 - Debian

1. Ativar SSL no Apache:

comando: `sudo a2enmod ssl`

`sudo a2ensite default-ssl`

`sudo systemctl reload apache2`

A terminal window showing the execution of commands to enable SSL on Apache. The user is 'user@vbox' and the shell is '\$'. The commands executed are 'sudo a2enmod ssl', 'sudo a2ensite default-ssl', and 'sudo systemctl reload apache2'. The output shows that the modules 'setenvif', 'mime', 'socache_shmcb', and 'ssl' are already enabled, and the site 'default-ssl' is also already enabled.

```
user@vbox:~$ sudo a2enmod ssl
sudo a2ensite default-ssl
sudo systemctl reload apache2
[sudo] password for user:
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled
Site default-ssl already enabled
user@vbox:~$
```

Observação: caso apareça um erro relacionado ao Apache, ignore - ele apenas indica que o Apache não estava rodando anteriormente.

2. Gerar certificado autoassinado (1 ano):

comando: `sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \`

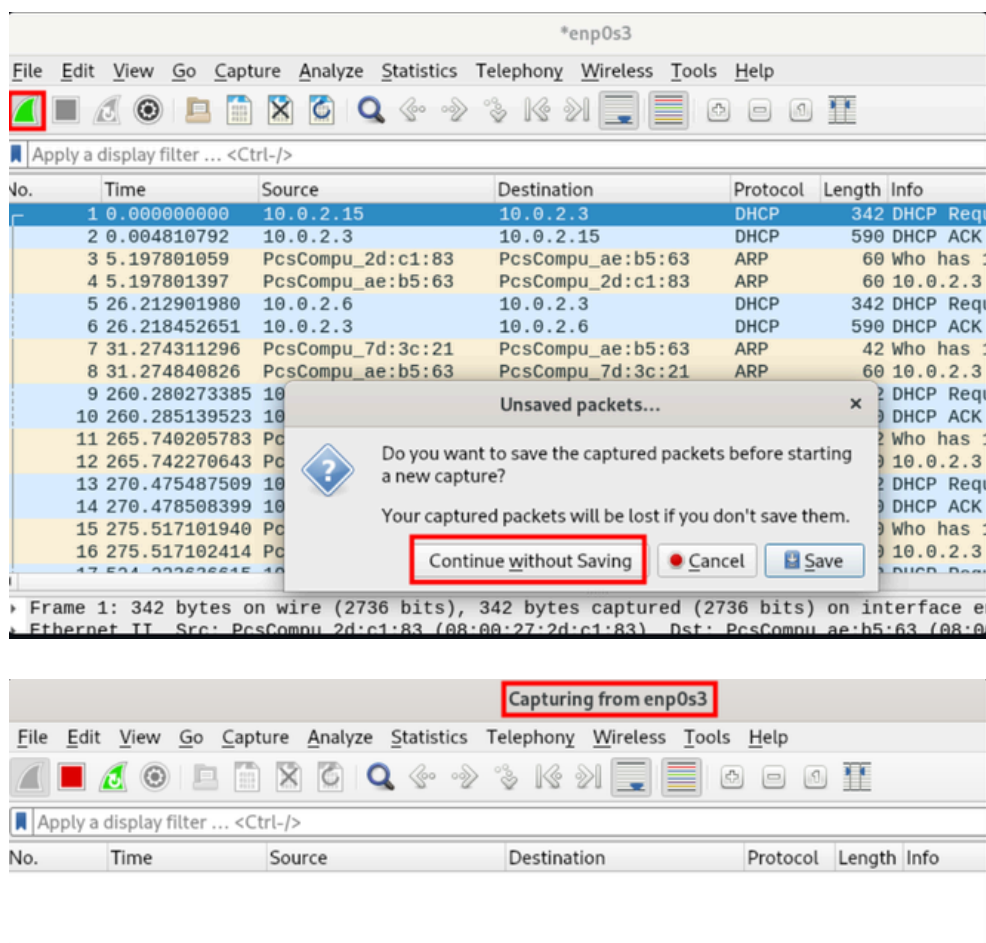
`-keyout /etc/ssl/private/apache.key \`

`-out /etc/ssl/certs/apache.crt \`

`-subj "/C=BR/ST=MG/L=Inatel/O=Lab/OU=NetSec/CN=<IP_USER1>"`

B) Reiniciar a captura e executar o teste HTTPS — user 2 - Debian

1. No Wireshark, reinicie a captura na interface da NAT Network para registrar somente o tráfego do novo teste.



3. Execute:

comando: `curl -vk https://<IP_USER1>`

O -k é intencional no laboratório: ele permite seguir com certificado autoassinado. Resultado esperado no terminal: a conexão HTTPS deve funcionar mesmo com o certificado autoassinado, e o curl deve retornar a resposta do Apache.

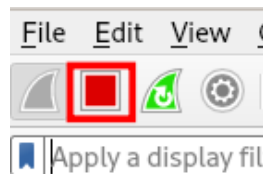
```

user@vbox:~$ curl -vk https://10.0.2.15
* Trying 10.0.2.15:443...
* Connected to 10.0.2.15 (10.0.2.15) port 443 (#0)
* ALPN: offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
* ALPN: server accepted http/1.1
* Server certificate:
*  subject: C=BR; ST=MG; L=Inatel; O=Lab; OU=NetSec; CN=<IP_USER1>
*  start date: May  8 00:29:03 2026 GMT
*  expire date: May  8 00:29:03 2027 GMT
*  issuer: C=BR; ST=MG; L=Inatel; O=Lab; OU=NetSec; CN=<IP_USER1>
*  SSL certificate verify result: self-signed certificate (18), continuing anyway.
* using HTTP/1.1
> GET / HTTP/1.1
> Host: 10.0.2.15
> User-Agent: curl/7.88.1
> Accept: */*
>
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* old SSL session ID is stale, removing
< HTTP/1.1 200 OK
< Date: Fri, 08 May 2026 00:34:34 GMT
< Server: Apache/2.4.67 (Debian)
< Last-Modified: Thu, 07 May 2026 23:06:49 GMT
< ETag: "18-6514255d209f2"
< Accept-Ranges: bytes
< Content-Length: 24
< Content-Type: text/html
<
<h1>HELLO_TLS_HTTP</h1>
* Connection #0 to host 10.0.2.15 left intact
user@vbox:~$

```

C) Observar no Wireshark

1. Pause a captura no Wireshark.



2. Remova o filtro anterior (se houver).

3. Aplique o filtro:

filtro: `tls && ip.addr == <IP_USER1>`

4. O que observar: pacotes de handshake TLS (ClientHello/ServerHello/Certificado) e payload cifrado.

tls && ip.addr == 10.0.2.6						
No.	Time	Source	Destination	Protocol	Length	Info
9	28.713867231	10.0.2.6	10.0.2.15	TLSv1.3	583	Client Hello
11	28.738747005	10.0.2.15				Server Hello, Change Cipher Spec, Application Data
13	28.762126592	10.0.2.6		Mark/Unmark Packet	Ctrl+M	
14	28.762911154	10.0.2.6		Ignore/Unignore Packet	Ctrl+D	
15	28.764128207	10.0.2.15				Application Data
16	28.764663114	10.0.2.15		Set/Unset Time Reference	Ctrl+T	
18	28.767408832	10.0.2.15		Time Shift...	Ctrl+Shift+T	
19	28.767819208	10.0.2.6				Application Data
21	28.771068796	10.0.2.15		Packet Comments		
Edit Resolved Name						
Apply as Filter						
Prepare as Filter						
Conversation Filter						
Colorize Conversation						
SCTP						
Follow						
TCP Stream						
UDP Stream						

Wireshark - Follow TCP Stream (tcp.stream eq 0) - enp0s3						
.....x...g...;E.Go...0						
*."6..c.....}.Y2.....1.....E2.vvd.a....0.						
..>.....0.....+./...\$.(.k.#.'g.						
...9.....3.....=<.5./...U.....						
.....h2.http/1.1.....1...						
*.(.....						
.....+.....3.&\$.9a....L1.....2.....<.....y.-.....						
.....Z...v...s.{.....=2...i.1.,^.....2...}.Y2.....1.....E2.vvd.a....0.						
.....+.....3.\$... .j.ITb.g.&...#J>...@.AJ.3S/.b.....&^.....u.'W...&...5...p/...@.....g\.=.....'\$..EX..1'..C..e.....k						
7R5...8...'7.dgA.Szj..._xX...x.....f...}..I.a.9..._a.s...C.....Mvt'k}						
.q.#..6..X,...i^c.Vg.h...4@.EX1...k+z7BR.%.....p%f-4.F/e.h...['[<H.W.jVe...#.n#Y...jv...=T.A.cB...W....Y....3...~....yOp..i...						
.H@.JFF.I						
2.jD#.7-...{...(.zK.....%...*v.hgv.{...^...P...;...I.r...et...b..R						
...4....sG-0...n..VGCF.E.Cz...#;...".r..C..B..).f...						
R.CW...g4...d)p...[A2.e.....?.....n..i#..H.U..k...!@...=.OD.../...1Dl.;&!q...+...l[;I.....a*.....jXEY.b.@.....7...((...ol						
((...aZ...i...!..5.sC.RA...#.....]H%...z.d..710.w...':.....IZ8...;0....%0.2.v.....yd...;@....C.....K.						
.B...f!..tf#...^..+..M.....(.=Zl.v...].V@A).u.[...d...#...!..H...t2...#...A>m...Gh...<.						
.fpSo)...?...?..Fh).B...o...j.OJ..w..cgF...w...#[...x.?G.LD...&...l...cE...mq...?F.S.x.b]Rdk&D.>.....{.....dNga...n...f.4,...						
...4...X!u...p...q...u...I...y.wh...=F.t...<...<...6pN...%. [0B.h.&...3...E.Mp....R....E%;...Z...}C...x.'U<.OI...Q...						
...a...a...9.Q...].e.q...d.m.w...UQ.g						
t..d.'o..b..B2...t...p..!\$.....b..w..._j...m".....ZAf..E.?...4						
.....I...J>..Ic.Egza..0.....[...~1d.u..X.;...v.						
.....^..w.....7..Sb.						
.....W...K.....;..'G.L...K....n...=.						
I-Y=.						
...=...f.						
.h!...G..TR..						
...bW.....%"+...f.y=...R...0.ax...E..6[K..?..a.						
ev...-...r.JcC.....vo.p.).\$.7..a..9..@.....T.....E...~...}1...-...v.C*.fIr.....o.:/...#8...v....@...S=b ...Ev...k.						
.....Z...l...j^...5.iJF...&...C.A.E...ew'...t...sMz..A5...t...X.....r.						
R.R+...&.Te...j...~...J...[...r.^...d...d...}{G.....vz.N..s>l.Fp0E...b.U...E..Z..So...!i..d...[...Ji..x:...Z.../=.../.						
hmik..y.=...p.i.J...3...[...].7..J.m.p...9.N..XK..7-M.u.....I.@...g.....9.....U...0...?0..7...lhI...".RZQ...=.sX...0...						
...4.wB.j...?"...5Y)...Lj#...q..T...S..B&a.y...b...=...dV.m7...K(.../.*...e...K.....-...!...X.....						
&...sf...&...L>4..O.N.[..._..R...+...K.K.Ls.HB4.*...Yp.....\D..1}.....\oj...F.....-...x...W#Z..H						

Ao final deste cenário: o conteúdo da aplicação não deve aparecer legível no Wireshark; apenas o handshake e os dados criptografados ficam visíveis.

VI. Verificação e Resultados

1) Checklist de sucesso do experimento

- Cenário HTTP (porta 80): curl -v http://<IP_USER1> retorna conteúdo e o Wireshark mostra GET/200 OK com payload legível.
- Cenário HTTPS (porta 443): curl -vk https://<IP_USER1> funciona, e no Wireshark aparecem ClientHello/ServerHello com payload cifrado.

2) Quadro comparativo

Protocolo	Porta	TLS	Visível no Wireshark	Payload
HTTP	80	Não	Métodos/URLs, cabeçalhos, corpo	Legível (HELLO_TLS_HTTP)
HTTPS	443	Sim	Handshake (ClientHello/ServerHello/Cert), Application Data	Illegível

3) Filtros práticos (copiar/colar)

- HTTP claro: http
- HTTPS: tls (ou tcp.port == 443)
- Handshake específico: tls.handshake
- Verificar string no tráfego claro: frame contains "HELLO_TLS_HTTP"

VII. Conclusão

Foi demonstrado, de forma prática, que a adoção de TLS em HTTP protege a confidencialidade das mensagens. Enquanto no tráfego sem TLS foi possível ler o payload em claro, no tráfego com TLS o Wireshark exibiu apenas metadados do handshake e pacotes cifrados.

Apêndice — Troubleshooting rápido

- Não vejo tráfego no Wireshark: selecione a interface da NAT Network correta e gere tráfego com curl no user 2 - Debian.
- HTTPS não sobe: confirme a2enmod ssl, a2ensite default-ssl, caminhos de certificado no default-ssl.conf e reinício do Apache.
- curl -vk https://<IP_USER1> falha: verifique se a porta 443 está em LISTEN (ss -lntp | grep :443).
- ping entre as VMs falha: confirme que ambas estão em NAT Network (NatNetwork), que os dois IPs estão na mesma faixa de rede e que o teste usa o endereço correto de destino.
- VMs não se enxergam: confirme ambas em NAT Network (NatNetwork) com DHCP ON.
- Wireshark não abre ou não captura corretamente: execute novamente com sudo wireshark e confirme que a interface da NAT Network foi selecionada.